You can verify that with this password and values of 20 and 30, the hashes computed by the preceding code are (for 20)

```
0xf330270724712 74d489ff841d4ea9e7e959a95c4d57d5f4f9c8541d474cb817a
```

and (for 30)

```
0xfaa88b88830698a2f37dd0fa4acbc258e126bc785f1407ba9824f408a905d784
```

Let’s apply these concepts to solve a new decentralized application.

# 5.4 Application of hashing

Let’s explore secure hashing for realizing privacy and security in a decentralized application. For this purpose, we’ll consider the blind auction problem described in the Solidity documentation (https://solidity.readthedocs.io/en/v0.5.3). As with the voting and ballot problems, the requirements of this problem have been altered significantly to enable us to focus on privacy and security issues.

> **PROBLEM STATEMENT:** A beneficiary plans a blind auction for a piece of artwork. There may be many pieces to be auctioned off, but for this problem, you’ll consider only one; you can always add others after this piece is sold. The beneficiary controls the various stages of the auction, `{Init, Bidding, Reveal, Done}`. After initiation by the beneficiary, the bidders bid one bid at a time during the `Bidding` phase, providing their bids securely and privately. Others, including the beneficiary, cannot see what each bid is. After a while, the beneficiary advances the stage to the `Reveal` phase. Now bidders openly send their bids, and the beneficiary opens the bids and identifies the highest bidder and highest bid. The beneficiary ends the auction by advancing the stage to Done, at which time the auction ends. The highest bid value is transferred to the beneficiary account. Nonwinner bidders can withdraw their deposits, and the winning bidder is returned the balance of their deposit.

Take a few minutes to review the problem statement and understand and play the steps in your mind or on paper before you proceed to design the solution.

## 5.4.1 Blind auction design